설계구조

1. 목표

코코 시스템은 하나의 공통 엔진을 유지하되, 사용 목적이 다른 두 실행면을 분리한다.

- 메인챗: 안정화된 결과를 소비하는 대화면
- 학습챗: 업로드 파일을 기준으로 분석, 검증, 병목 수정이 이루어지는 실험면

핵심 원칙은 엔진을 둘로 나누지 않는 것이다. 공통 코어는 하나만 유지하고, 메인챗과 학습챗은 같은 엔진의 다른 소비자 역할을 맡는다.

2. 대원칙

제1원칙

파일은 반드시 코코 앱에 먼저 등록한 후 엔진을 통해 분석한다.

제2원칙

앱 처리 경로와 내부 엔진 경로는 분리하지 않고 하나의 공통 경로로 통합한다.

제3원칙

코코 앱에 등록된 자료를 기준으로 내부 엔진을 학습시키고 오류를 수정한다.

제4원칙

문제 흐름에 병목이 생기면 완전히 해결한 후 다음 문제로 넘어간다.

3. 공통 코어

공통 코어는 하나의 분석 파이프라인으로 유지한다.

```
이미지 입력
→ OCR 문자/좌표 추출
→ 수식 영역 탐지
→ 1차 규칙 복원
→ 필요 시 pix2tex fallback
→ LaTeX 정제
→ SymPy용 정규화
→ SymPy 파싱/검산
→ 후보 랭킹
→ 최종 수식 채택
```

코어 구성은 아래와 같이 유지한다.

- OCR 레이어: 문자와 bbox 추출
- formula cropper: 수식 영역만 잘라냄
- tsv_reconstructor: 좌표 기반 위첨자/분수/연산자 구조 복원
- pix2tex_manager: fallback 수식 OCR
- latex_cleaner: 수식 OCR 출력 노이즈 제거
- latex_to_sympy_normalizer: SymPy 친화 표현 변환
- sympy_validator: 파싱 가능 여부, 계산 가능 여부, 선택지 일치 여부 확인
- candidate_ranker: 여러 후보 중 최종 채택

4. 메인챗과 학습챗 분리

메인챗

메인챗은 안정화된 결과만 보여주는 실행면이다.

- 일반 대화
- 요약된 결과 소비
- 실험 로그 비노출
- 검증되지 않은 후보 비노출
- 학습챗에서 verified 된 결과만 참조

학습챗

학습챗은 업로드 파일 중심의 분석과 수정이 이루어지는 실행면이다.

- 파일 업로드
- 분석 파이프라인 실행
- 중간 로그 확인
- OCR/pix2tex/SymPy 병목 추적
- 회귀 테스트
- verified 결과를 메인챗으로 승격

5. 저장 구조

저장 구조는 메인과 학습을 분리하되, 엔진 결과는 공통 스키마를 유지한다.

- main

- main_chat_history

- promoted_summaries

- study

- documents

- study_chat_history

- analysis artifacts

- debug logs

권장 경로 예시:

- `~/Library/Application Support/CocoAIStudy/main`
- `~/Library/Application Support/CocoAIStudy/study`

6. 승격 규칙

학습챗의 결과가 아래 조건을 만족할 때만 메인챗으로 승격한다.

- `validation_status == verified`
- `chosen_formula` 존재
- `matched_choice` 또는 최종 답 존재
- parse 성공 후보 기반 결과

아래 상태는 메인챗으로 승격하지 않는다.

- 보강 필요
- failed
- fallback_failed_only
- parse 실패 후보만 있는 상태

7. 실행 흐름

메인챗 실행 흐름

메인챗은 직접 분석을 수행하지 않는다.

사용자 대화

→ 메인 상태 조회

→ 학습챗에서 승격된 결과 요약 소비

→ 응답

학습챗 실행 흐름

학습챗은 업로드 파일을 기준으로 분석을 수행한다.

파일 등록
→ 공통 분석 파이프라인 실행
→ OCR / 수식 OCR / 정제 / 파싱 / 검산 / 랭킹
→ 중간 로그 저장
→ verified 여부 판정
→ 필요 시 엔진 수정
→ 재업로드 후 재검증

---

8. 동시성 및 작업 제어

같은 입력에 대해 중복 분석이 발생하지 않도록 task 레이어를 둔다.

- task_id 기준 mutex/lock
- 상태 머신:

- queued
- running
- succeeded
- failed
- retry_wait

- running 상태 동일 task 재실행 금지
- idempotency key 기반 요청 합치기
- 재시도는 최대 1~2회, 지수 백오프 적용
- UI는 상태 조회만 수행하고 실제 실행은 worker가 담당

로그 항목:

- task_id
- retry_count
- state_transition
- lock_acquired

9. 현재 개발 운영 원칙

앞으로의 개발은 아래 순서로 진행한다.

1. 코코 앱에 파일 등록
2. 앱이 만든 실제 결과와 저장 로그 확인
3. 병목 구간 확인
4. 공통 코어 수정
5. 같은 앱 경로로 재검증
6. 문제가 완전히 해결된 뒤 다음 문제로 이동

10. 결론

코코의 구조는 `공통 엔진 하나 + 메인챗/학습챗 두 실행면`으로 정리한다.

- 엔진은 하나로 유지한다.
- 학습챗은 실험, 수정, 검증을 담당한다.
- 메인챗은 안정화된 결과만 소비한다.
- 앱 업로드 경로를 기준으로만 학습하고 수정한다.

이 설계를 기준으로 이후 개발은 메인과 학습의 역할을 분리하되, 코어 분석 파이프라인은 한 군데에서만 유지·개선한다.